Experiment No 2

Title:

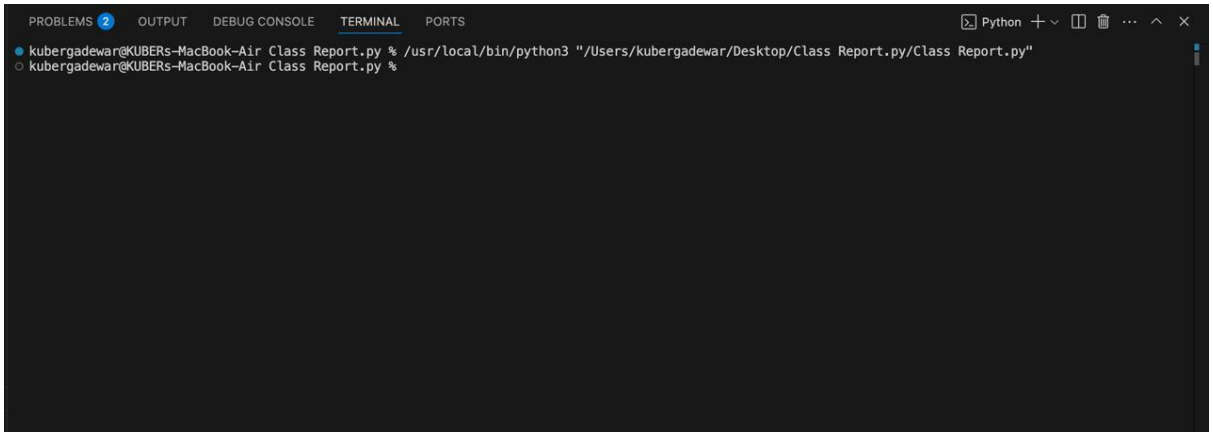
Design a Dynamic Report Generator in Python that uses decorators, class methods, and magic methods to customize and format reports. The system should allow users to define report templates and apply various formatting options dynamically.

Source Code:

```
def uppercase_decorator(func):
    """A decorator that converts the string output of a method to uppercase."""
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result.upper()  # Convert result to uppercase
    return wrapper


class Report:
    def init(self, title):
        self.title = title


    @classmethod
    def from_template(cls, template):
        """Class method to instantiate the object using a template string."""
        return cls(template)  # Create object from a template string


    def str(self):
```

```
        """Magic method to define the string representation of the object."""
        return f"Report Title: {self.title}"

    @uppercase_decorator
    def generate(self):
        """Generates the report, automatically formatted to uppercase via decorator."""
        return f"This is the report: {self.title}"
```

Output Screenshort:



```
# --- Verification / Example Usage ---

if _name_ == "main":

    # 1. Creating a report using the standard constructor

    report1 = Report("Quarterly Financials")

    print("--- Using standard str magic method ---")

    print(report1)

    print("--- Using decorated generate method ---")

    print(report1.generate())


    print("\n" + "="*40 + "\n")


    # 2. Creating a report dynamically via the class
method template

    report2 = Report.from_template("Annual Performance
Review")
```

```
print("--- Using classmethod from_template ---")

print(report2)

print("--- Using decorated generate method ---")

print(report2.generate())
```